Le projet et les méthodes

Personne 1 | Le besoin et les trois premières méthodes.

Environ 8 minutes à deux. Texte fondé sur l'énoncé du mini-projet. Les titres et indications en gris ne sont pas à lire. Si vous présentez seul, retirez le passage de parole.

1. Le projet et l'analyse du besoin - environ 1 minute

Bonjour. Le sujet nous demande de créer un annuaire en C, puis de comparer quatre façons d'y rechercher un e-mail. On compare à la fois leur complexité théorique et leur temps réel.

Pour décrire le besoin, on reprend les cinq points de la grille IOCSF.

Les données sont des utilisateurs avec un e-mail et un identifiant entier, ainsi que l'e-mail à rechercher. On a regroupé les informations d'un utilisateur dans une structure appelée **User**.

La sortie d'une recherche est un booléen : vrai si l'e-mail existe, faux sinon.

Les contraintes sont d'utiliser le C, de garder la même interface pour les méthodes, de gérer l'annuaire vide et de libérer la mémoire.

Le volume testé va de 100 à un million d'utilisateurs.

Enfin, la fréquence de recherche dépend de l'usage : le sujet va d'un petit annuaire consulté deux fois par jour à un service interrogé des milliers de fois par seconde.

2. Les trois premières méthodes - environ 2 minutes

La première méthode utilise un tableau alloué dynamiquement. On ajoute les utilisateurs à la fin et on agrandit le tableau quand il est plein.

Pour rechercher un e-mail, on parcourt les cases une par une avec une boucle. Si l'e-mail est absent, on doit toutes les vérifier. La recherche est donc en O de n dans le pire cas, avec n le nombre d'utilisateurs.

La deuxième méthode est une table de hachage. Une fonction transforme l'e-mail en numéro de case. On cherche directement dans cette case, plutôt que dans tout l'annuaire.

Deux e-mails peuvent donner la même case : c'est une collision. Notre code les garde dans une liste attachée à cette case, puis compare leurs e-mails pour trouver le bon.

La recherche est en moyenne en O de 1 si les données sont bien réparties et la table assez grande. Mais au pire, si tous les utilisateurs tombent dans la même case, elle est en O de n.

La troisième méthode utilise un tableau qui reste trié par e-mail. On regarde la case du milieu, puis on garde la moitié où l'e-mail peut se trouver. On recommence : c'est la recherche dichotomique, en O de log n. Une vingtaine d'étapes suffisent pour un million d'utilisateurs.

Pour l'insertion, on a choisi d'ajouter directement à la bonne place. Le code part de la fin et décale les utilisateurs jusqu'à libérer cette place. On ne retrie donc pas tout le tableau. Cette insertion coûte O de n dans le pire cas, car on peut devoir déplacer tous les utilisateurs.

La liste, les tests et les mesures

Fin de la personne 1, puis début de la personne 2.

3. La quatrième méthode, les tests et la mémoire - environ 1 minute

Le sujet ajoute une quatrième méthode : une liste chaînée indépendante. Chaque utilisateur est dans un nœud alloué avec **malloc**, qui contient aussi l'adresse du suivant.

On insère au début, en O de 1. Pour rechercher, on suit les liens un par un : on peut tout parcourir, donc c'est O de n au pire.

Montrer les fonctions déclarées dans annuaire.h.

Les quatre versions ont les mêmes opérations d'ajout et de recherche. On les teste séparément : annuaire vide, trois utilisateurs présents, un e-mail absent et recherche après nettoyage.

Pour nettoyer, on libère chaque tableau et chaque nœud des listes. On remet ensuite les variables à zéro pour pouvoir réutiliser les annuaires.

Je laisse maintenant la suite à mon binôme pour les mesures et leur analyse.

4. Le protocole de mesure - environ 1 minute

Comme demandé, on teste 100, mille, dix mille, cent mille et un million d'utilisateurs. Les quatre structures reçoivent exactement les mêmes données.

Les e-mails sont générés automatiquement à partir d'un compteur. On ajoute des zéros devant les numéros pour garder l'ordre alphabétique. Le tableau trié se remplit ainsi sans décalages pendant cette étape.

On recherche ensuite un e-mail absent, comme le demande le sujet. Cela force le tableau non trié et la liste à aller jusqu'au bout. Pour le hachage, cela ne provoque pas forcément le pire cas des collisions.

On utilise **clock**, de la bibliothèque **time.h**. On répète les recherches, on mesure la durée totale et on divise par leur nombre. Le remplissage est fait avant le chronométrage. Les temps moyens sont affichés et enregistrés dans un CSV, avec une colonne pour chaque méthode.

Les résultats et les choix

Personne 2 | Analyse et réponses aux quatre situations du sujet.

5. Les résultats et leur explication - environ 1 minute 30

Afficher le tableau des résultats du programme.

Les résultats vont globalement dans le sens attendu. Quand on passe de cent mille à un million d'utilisateurs, la recherche séquentielle passe d'environ une milliseconde à 12 millisecondes. Parcourir tout l'annuaire devient donc plus coûteux quand il grandit.

À un million, le hachage et la dichotomie restent sous la microseconde dans notre test. Une microseconde vaut un millième de milliseconde. Ces méthodes évitent de vérifier tous les utilisateurs.

La liste chaînée prend environ 57 millisecondes, soit près de cinq fois le temps du tableau non trié. Pourtant, les deux recherches sont en O de n.

Le tableau stocke les utilisateurs côte à côte, ce qui favorise le cache, une petite mémoire rapide du processeur. Dans la liste, les nœuds sont alloués séparément et on doit suivre leurs adresses. Cela peut expliquer une partie de l'écart.

Le coût des opérations, la mémoire et l'environnement jouent donc aussi sur les temps. Nos chiffres viennent d'une seule exécution avec le même e-mail absent répété. Ils ne prouvent pas qu'une méthode gagne dans tous les cas, ni que le pire cas du hachage a été mesuré.

6. Quelle méthode choisir selon le sujet ? - environ 1 minute

Pour les 30 employés avec deux consultations par jour, on choisirait le tableau non trié : il est simple et suffisant pour cette taille.

Pour cinq millions d'utilisateurs et des milliers de recherches par seconde, on choisirait le hachage, avec une table adaptée au volume. Les 100 003 cases de notre code ne sont pas un choix à reprendre automatiquement.

Pour un fichier déjà trié, chargé une fois et jamais modifié, on choisirait la dichotomie. On profite d'une recherche rapide sans payer le coût des décalages lors d'ajouts.

Pour des inscriptions nombreuses et continues, en plus des recherches, on choisirait aussi le hachage, en surveillant les collisions quand l'annuaire grossit.

Ce projet montre donc que le bon choix dépend du volume, des recherches et des ajouts. La complexité aide à prévoir le travail nécessaire, et les mesures montrent aussi l'effet de l'organisation des données en mémoire.